

Chapter 8 --- Keys and Constraints

Keys and constraints are the rules that keep data in a relational database accurate, unique, and properly connected. Without them, a table would allow duplicate records, missing values, and broken relationships between tables.

8.1 Understanding Keys and Constraints

Definition: A constraint is a rule applied to a column (or a set of columns) that restricts the kind of data that can be stored in it, in order to maintain accuracy and reliability. A key is a column (or combination of columns) used to uniquely identify rows or to connect one table to another.

In short: constraints protect data quality; keys identify and connect records.

Types of Keys

Key Description Example

Candidate Key Any column (or set of StudentID, Email ---
columns) that could both could identify a
qualify to uniquely student
identify a row

Primary Key The candidate key StudentID
actually chosen to
uniquely identify each
row in the table

Composite Key A primary key made up (OrderID, ProductID) in
of two or more columns an OrderItems table
together

Unique Key A column that must hold Email, PhoneNumber
unique values, but is
not the primary key

Foreign Key A column in one table StudentID in the Marks
that refers to the table, referencing
primary key of another Students
table, creating a
relationship

Types of Constraints --- Overview

Constraint Purpose

PRIMARY KEY Uniquely identifies each row;
combines NOT NULL + UNIQUE

NOT NULL Column cannot store an empty (NULL)
value

UNIQUE All values in the column must be
different from each other

DEFAULT Sets an automatic value when none
is provided

CHECK Restricts values based on a
condition

AUTO_INCREMENT Automatically generates the next
number for a column

FOREIGN KEY Links a column to the primary key
of another table

8.2 Using PRIMARY KEY, NOT NULL and UNIQUE

PRIMARY KEY

A PRIMARY KEY uniquely identifies every row in a table. MySQL automatically enforces two rules on it: values must be unique, and values cannot be NULL. A table can have only one primary key.

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY, StudentName VARCHAR(50));
```

A primary key can also be added after the table is created, or defined on multiple columns (a composite key):

```
ALTER TABLE Students ADD PRIMARY KEY (StudentID); -- composite primary key
```

```
CREATE TABLE OrderItems ( OrderID
```

NOT NULL

Ensures a column can never be left empty --- every row must supply a value for it.

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY, StudentName VARCHAR(50) NOT NULL);
```

Without NOT NULL, a column silently accepts NULL, which can break calculations, searches, and joins later on.

UNIQUE

Ensures every value in a column is different from every other value in that column --- but, unlike PRIMARY KEY, a UNIQUE column is allowed to store one NULL value (since NULL is not considered equal to anything, including another NULL).

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY, StudentName VARCHAR(50) NOT NULL, Email VARCHAR(50) UNIQUE);
```

A table can have many UNIQUE columns, but only one PRIMARY KEY.

8.3 Using DEFAULT, CHECK and AUTO_INCREMENT

DEFAULT

Supplies an automatic value for a column when no value is given during an INSERT.

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY, StudentName VARCHAR(50) NOT NULL, Branch VARCHAR(50) DEFAULT 'CSE' );
```

If a Branch is not supplied on insert, MySQL automatically stores 'CSE'.

CHECK

Restricts the values allowed in a column based on a logical condition.

(Supported from MySQL 8.0.16 onward.)

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY, Age INT CHECK (Age >= 17), Marks INT CHECK (Marks <= 100) );
```

Any INSERT or UPDATE that violates the condition is rejected by MySQL.

AUTO_INCREMENT

Automatically generates the next whole number for a column --- most commonly used with a primary key so you never have to supply an ID manually.

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY AUTO_INCREMENT, StudentName VARCHAR(50) NOT NULL);
```

- Only one AUTO_INCREMENT column is allowed per table.
- It must be indexed --- normally as the PRIMARY KEY.
- The counter keeps increasing even if rows are deleted (it does not reuse deleted numbers), unless the table is truncated.

8.4 Comparing PRIMARY KEY and UNIQUE Constraints

PRIMARY KEY and UNIQUE are often confused because both prevent duplicate values --- but they are not interchangeable.

	PRIMARY KEY	UNIQUE
Feature	Yes	Yes

Allows NULL values No Yes --- one NULL is allowed

Number allowed per table Only one Multiple

Main purpose Uniquely identifies each row Prevents duplicate values in a column

Automatically indexed Yes (clustered index in InnoDB) Yes (unique index)

Can be referenced by a FOREIGN KEY Yes Yes

Typical use StudentID, OrderID Email, PhoneNumber,

UserName

Rule of thumb: use PRIMARY KEY for the one column (or column combination) that identifies each row of the table itself. Use UNIQUE for any other column that must not repeat, such as an email address or a username, where the value is still a real-world identifier but not the table's main key.

Putting It All Together --- A Complete Example

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY AUTO_INCREMENT, StudentName VARCHAR(50) NOT NULL, Email VARCHAR(100) NOT NULL, Branch VARCHAR(50) DEFAULT 'CSE', Age INT CHECK (Age >= 17), JoinedOn DATE DEFAULT (CURRENT_DATE))
```

Column Constraint(s) What It Guarantees

StudentID PRIMARY KEY, Unique, non-null,
AUTO_INCREMENT auto-generated ID for
every student

StudentName NOT NULL Every student must have
a name on record

Email UNIQUE No two students can
share the same email

Branch DEFAULT 'CSE' Branch is filled in
automatically if not
provided

Age CHECK (Age >= 17) Rejects any age below
17

JoinedOn DEFAULT (CURRENT_DATE) Automatically records
today's date if not
provided

■ Final Takeaway Keys identify records and connect tables --- PRIMARY KEY for a table's own identity, FOREIGN KEY for relationships, UNIQUE for other must-not-repeat values. Constraints protect data quality --- NOT NULL stops missing values, CHECK stops invalid values, DEFAULT fills in sensible values automatically, and AUTO_INCREMENT removes the need to generate IDs manually. Used together, they make a database trustworthy by design, not just by convention.

8.5 Interview Questions --- Chapter 8

Q1. What is the difference between a primary key and a unique key?

Ideal answer: A primary key uniquely identifies each row, cannot contain NULL, and a table can have only one. A unique key also enforces uniqueness but allows one NULL value, and a table can have several

unique keys.

Q2. Can a table have more than one primary key?

Ideal answer: No --- a table can have only one primary key, though that primary key can be composite (made of more than one column).

Q3. What is the difference between NOT NULL and UNIQUE?

Ideal answer: NOT NULL only ensures a column cannot be empty; it does not prevent duplicate values. UNIQUE ensures no two rows share the same value in that column, but (unlike NOT NULL) it still allows one NULL.

Q4. What does AUTO_INCREMENT do, and where is it usually used?

Ideal answer: AUTO_INCREMENT automatically generates the next sequential number for a column, most commonly the primary key, so IDs never need to be entered manually. Only one AUTO_INCREMENT column is allowed per table, and it must be indexed.

Q5. What is a foreign key?

Ideal answer: A foreign key is a column in one table that refers to the primary key of another table, creating a relationship between the two tables and preventing values that don't exist in the referenced table.

Interview Questions — Chapter 8

Q1. Can a table have two primary keys?

Ideal answer: No. A table has one primary key constraint, although that key may be composite and contain multiple columns.

Q2. PRIMARY KEY vs UNIQUE?

Ideal answer: PRIMARY KEY uniquely identifies rows and cannot be NULL; UNIQUE enforces uniqueness and can allow NULL according to MySQL's rules.

Q3. What does NOT NULL enforce?

Ideal answer: It prevents a column from storing NULL values.

Q4. What is a foreign key?

Ideal answer: It is a column or set of columns that references a key in another table and enforces referential integrity.

Q5. Why are constraints important?

Ideal answer: They enforce data-quality rules at the database level instead of relying only on application code.